

EE284A Lab 1 Deliverables

Edward Bayliss

Section 4: Analog Temperature Sensor

Deliverable 1: TMP36 Datasheet Screenshots

TO-92 package pinout

Figure 3. R-8 (SOIC_N)

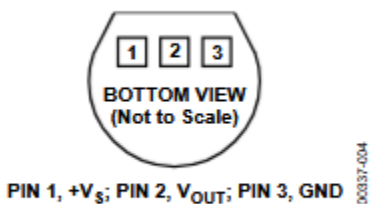


Figure 4. T-3 (TO-92)

Operating voltage range

Output Voltage Range | 100 | 2000 | mV

Output voltage vs temperature graph

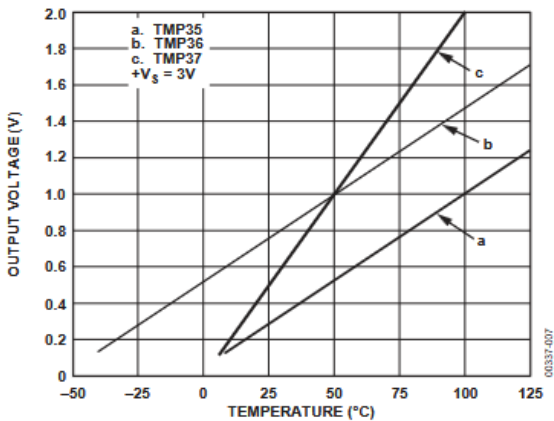


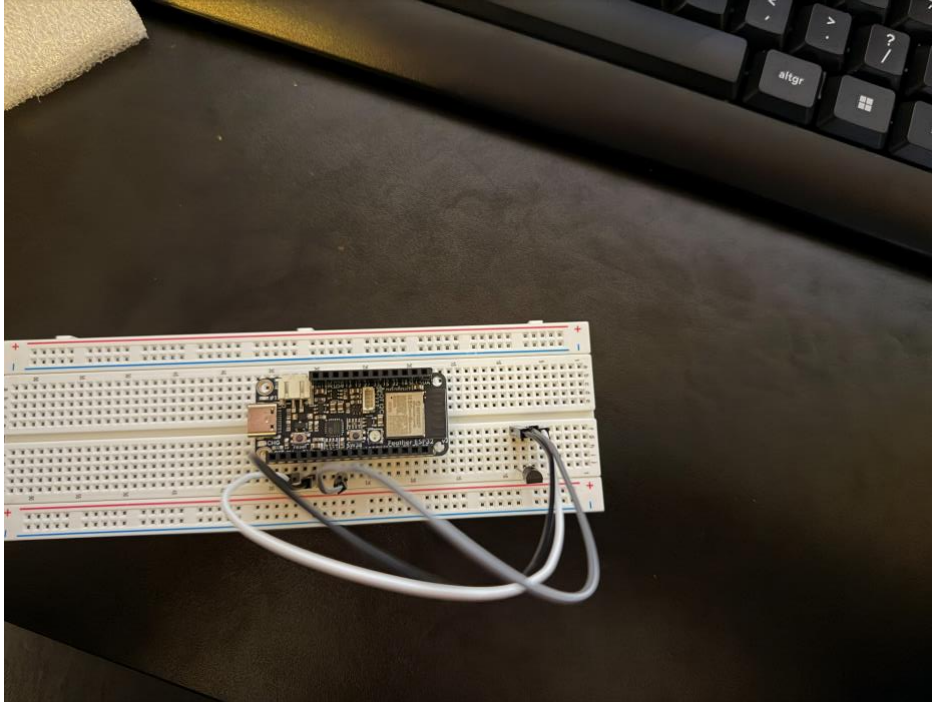
Figure 6. Output Voltage vs. Temperature

Scaling table

Table 4. TMP35/TMP36/TMP37 Output Characteristics

Sensor	Offset Voltage (V)	Output Voltage Scaling (mV/°C)	Output Voltage at 25°C (mV)
TMP35	0	10	250
TMP36	0.5	10	750
TMP37	0	20	500

Deliverable 2: TMP36 Wiring Photo



Deliverable 3: Temperature Conversion Equation

The ESP32 ADC is 12-bit with $V_{ref} = 2.1V$ at 6dB attenuation, giving values from 0 to 4095.

DN to voltage:

$$V = (DN / 4095) * 2.1$$

Voltage to temperature (from TMP36 datasheet: 750mV at 25 degrees Celsius, 10mV/degree scale):

$$T(C) = (V - 0.5) / 0.010$$

Combined:

$$T(C) = ((DN / 4095) * 2.1 - 0.5) / 0.010$$

Deliverable 4: Serial Output Screenshots

```
The raw ADC value is 1266, which converts to 0.6492 V. The temperature is below 23C, in fact it is 14.92 C
The raw ADC value is 1267, which converts to 0.6497 V. The temperature is below 23C, in fact it is 14.97 C
The raw ADC value is 1282, which converts to 0.6574 V. The temperature is below 23C, in fact it is 15.74 C
The raw ADC value is 1283, which converts to 0.6579 V. The temperature is below 23C, in fact it is 15.79 C
The raw ADC value is 1284, which converts to 0.6585 V. The temperature is below 23C, in fact it is 15.85 C
The raw ADC value is 1281, which converts to 0.6569 V. The temperature is below 23C, in fact it is 15.69 C
The raw ADC value is 1275, which converts to 0.6538 V. The temperature is below 23C, in fact it is 15.38 C
The raw ADC value is 1271, which converts to 0.6518 V. The temperature is below 23C, in fact it is 15.18 C
The raw ADC value is 1267, which converts to 0.6497 V. The temperature is below 23C, in fact it is 14.97 C
The raw ADC value is 1270, which converts to 0.6513 V. The temperature is below 23C, in fact it is 15.13 C
The raw ADC value is 1299, which converts to 0.6662 V. The temperature is below 23C, in fact it is 16.62 C
The raw ADC value is 1242, which converts to 0.6369 V. The temperature is below 23C, in fact it is 13.69 C
The raw ADC value is 1284, which converts to 0.6585 V. The temperature is below 23C, in fact it is 15.85 C
The raw ADC value is 1277, which converts to 0.6549 V. The temperature is below 23C, in fact it is 15.49 C
The raw ADC value is 1271, which converts to 0.6518 V. The temperature is below 23C, in fact it is 15.18 C
The raw ADC value is 1267, which converts to 0.6497 V. The temperature is below 23C, in fact it is 14.97 C
The raw ADC value is 1285, which converts to 0.6590 V. The temperature is below 23C, in fact it is 15.90 C
The raw ADC value is 1279, which converts to 0.6559 V. The temperature is below 23C, in fact it is 15.59 C
The raw ADC value is 1277, which converts to 0.6549 V. The temperature is below 23C, in fact it is 15.49 C
The raw ADC value is 1279, which converts to 0.6559 V. The temperature is below 23C, in fact it is 15.59 C
The raw ADC value is 1296, which converts to 0.6646 V. The temperature is below 23C, in fact it is 16.46 C
The raw ADC value is 1271, which converts to 0.6518 V. The temperature is below 23C, in fact it is 15.18 C
```

Ln 45, Col 45 Adafruit Feather ESP32 V2 on COM3 2

Deliverable 5: Complete TMP36 Arduino Script

```
1  #define ADC_PIN A2
2  int led = LED_BUILTIN;
3
4  /* LED Variables */
5  const unsigned long IntervalHigh = 500; // ms
6  const unsigned long IntervalLow = 500; // ms
7  unsigned long ledInterval = 0;
8  unsigned long lastChangeTime = 0;
9  bool ledIsHigh = false;
10
11 /* ADC Variables*/
12 unsigned long lastAdcTime = 0;
13 const unsigned long adcInterval = 500; // ms
14 uint16_t mvRead = 0;
15 float celciusRead = 0;
16
17 const float DvDc = 100;
18 const float MvOffset = 0.5;
19
20
21 void setup() {
22   // Some boards work best if we also make a serial connection
23   Serial.begin(115200);
24
25   // set LED to be an output pin
26   pinMode(led, OUTPUT);
27
28   analogReadResolution(12);
29
30   analogSetPinAttenuation(ADC_PIN, ADC_6db); // lock attenuation
31   delay(50);
32 }
33
34 void loop() {
35   unsigned long currentTime = millis();
36
37   if(currentTime - lastAdcTime >= adcInterval) {
38     analogSetPinAttenuation(ADC_PIN, ADC_6db);
39     delay(50);
40
41     lastAdcTime = currentTime;
42     mvRead = analogRead(ADC_PIN);
43
44     float voltage = (mvRead / 4095.0f) * 2.1f;
45     celciusRead = (voltage - 0.5f) / 0.010f;
46
47     Serial.printf(
48       "The raw ADC value is %u, which converts to %.4f V. "
49       "The temperature is %s 23C, in fact it is %.2f C\n",
50       mvRead,
51       voltage,
52       (celciusRead > 23.0f) ? "above" : "below",
53       celciusRead
54     );
55   }
56
57   ledRoutine(currentTime);
58 }
59 }
```



Deliverable 6: Max, Min, and Delta Temperature over 1 Minute

The maximum temperature was 16.41 degrees Celsius and the minimum temperature was 16.12 degrees Celsius, with a delta of 0.29 degrees Celsius.

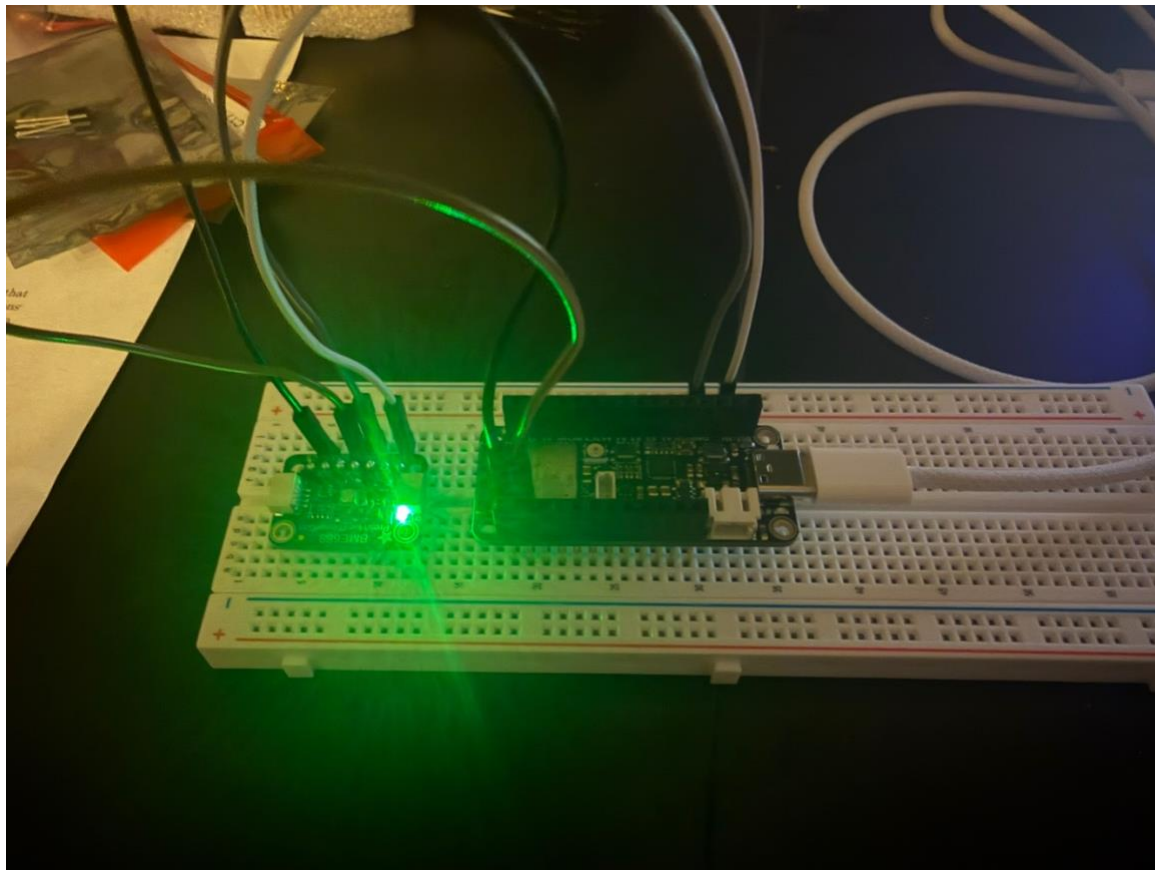
Deliverable 7: Another Temperature Sensor Type

Thermocouple (Type K)

When two dissimilar metals are joined at a junction, a temperature-dependent voltage of $\sim 41\mu\text{V}/\text{degree Celsius}$ is generated via the Seebeck effect. This voltage is too small for an ADC to read directly, so an amplifier IC (e.g. MAX31855) boosts it and does cold-junction compensation before sending the result to the MCU over SPI. The main advantage over the TMP36 is its range of -200 to 1350 degrees Celsius vs -40 to 125 degrees Celsius, which makes thermocouples the standard choice for high-temperature applications like furnaces and engines.

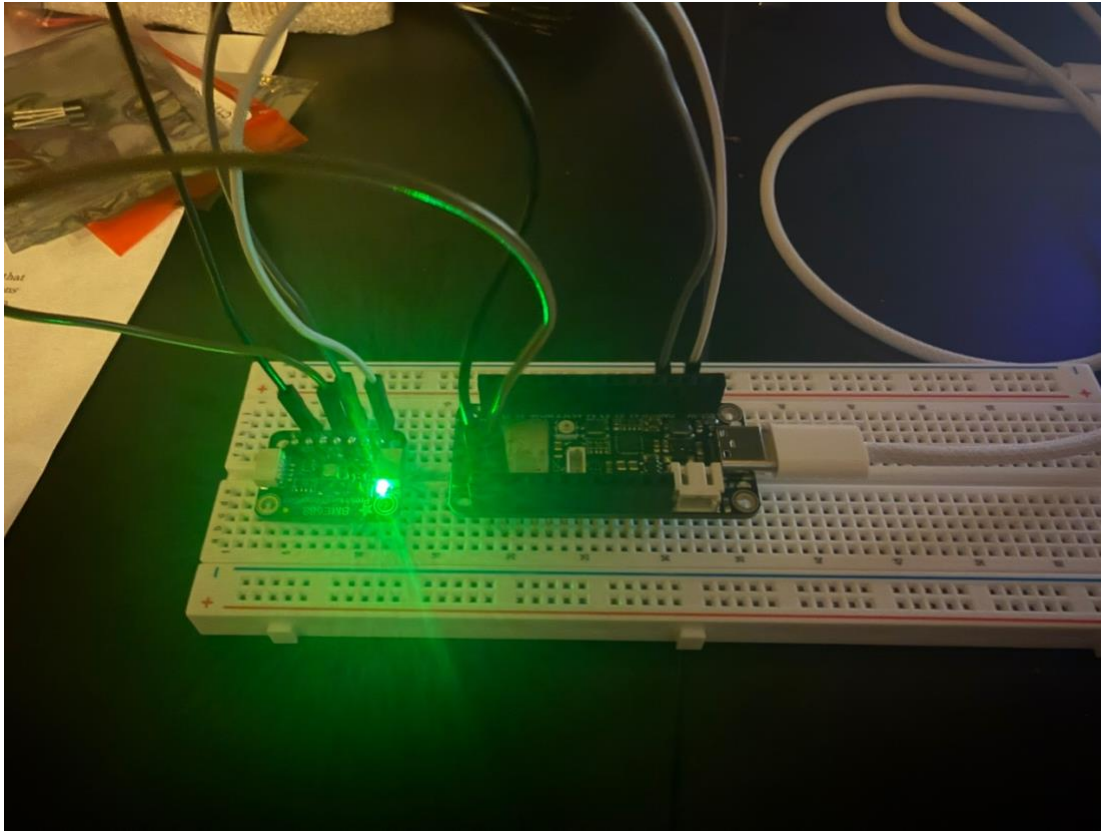
Section 5: I2C and SPI

Deliverable 8: I2C Wiring and Serial Output



21:40:38.729 -> Temperature = 21.12 *C

Deliverable 9: SPI Wiring and Serial Output



07:00:51.456 -> Temperature = 19.46 *C

The data over SPI looked the same as over I2C. Both protocols are reading from the same sensor registers so the temperature, humidity, pressure and gas values were consistent. Any small differences between readings were just sensor noise between samples.

Deliverable 10: I2C vs SPI Transaction Order

I2C uses two wires (SDA and SCL) and supports multiple devices on the same bus using addressing:

- Master pulls SDA low while SCL is high to signal a START
- Master sends the 7-bit device address and a read/write bit
- The addressed device pulls SDA low to ACK
- Data bytes transfer one at a time, each followed by an ACK
- Master signals STOP by letting SDA go high while SCL is high

SPI uses four wires (MOSI, MISO, SCK, CS) and is faster but point-to-point:

- Master pulls CS low to select the device
- Master toggles SCK to clock data

- Data transfers in both directions at once: master out on MOSI, device out on MISO, one bit per clock edge
- Master pulls CS high to end the transaction

The main difference is that I2C is half-duplex and uses ACKs to confirm each byte, while SPI is full-duplex with no acknowledgement. SPI is faster but needs an extra CS wire for each device. I2C can share two wires across many devices using addressing.

Deliverable 11: BME688 Max and Min Temperature over 1 Minute

The maximum and minimum temperatures observed over 1 minute were 19.63 and 19.46 degrees Celsius, with a delta of 0.17 degrees Celsius. This delta is smaller than the TMP36's delta of 0.29 degrees Celsius. The BME688 has a built-in ADC with finer resolution and lower noise than the ESP32's general-purpose ADC used by the TMP36, which accounts for the difference.

Section 6: Signal Processing

Deliverable 12: Filter Implementation Screenshots with BME688 via SPI

Moving average filter code

```

134 // Moving average (small)
135 maSumSmall -= maBufSmall[maHeadSmall];
136 maBufSmall[maHeadSmall] = raw;
137 maSumSmall += raw;
138 maHeadSmall = (maHeadSmall + 1) % N_SMALL;
139 if (maCountSmall < N_SMALL) maCountSmall++;
140 float ma50 = maSumSmall / maCountSmall;

```

Moving median filter code

```

150 // Moving median (small)
151 medBufSmall[medHeadSmall] = raw;
152 medHeadSmall = (medHeadSmall + 1) % N_SMALL;
153 if (medCountSmall < N_SMALL) medCountSmall++;
154 float mm50 = getMedian(medBufSmall, medHeadSmall, medCountSmall, N_SMALL);

```

```

68 float getMedian(float* buf, int head, int count, int size) {
69     // Copy window into sort buffer
70     for (int i = 0; i < count; i++) {
71         sortBuf[i] = buf[(head - count + i + size) % size];
72     }
73     // Simple insertion sort
74     for (int i = 1; i < count; i++) {
75         float key = sortBuf[i];
76         int j = i - 1;
77         while (j >= 0 && sortBuf[j] > key) {
78             sortBuf[j + 1] = sortBuf[j];
79             j--;
80         }
81         sortBuf[j + 1] = key;
82     }
83     return sortBuf[count / 2];
84 }

```

Exponential moving average code

```

162 // EMA
163 emaSmall = alphaSmall * raw + (1.0 - alphaSmall) * emaSmall;
164 emaLarge = alphaLarge * raw + (1.0 - alphaLarge) * emaLarge;
165

```

Deliverable 13: Filter Comparison Table (5000 samples)

Method	Min (°C)	Max (°C)	Mean (°C)	StdDev (°C)
Raw	19.278	21.622	19.713	0.2594
MovAvg N=50	19.285	21.622	19.712	0.2681
MovAvg N=500	19.304	21.622	19.679	0.2608
MovMed N=50	19.283	21.622	19.711	0.2593
MovMed N=500	19.288	21.622	19.674	0.2611
EMA $\alpha=2/51$	19.292	21.622	19.721	0.2728
EMA $\alpha=2/501$	19.430	21.622	19.789	0.3249

Deliverable 14: Observations on Metric Changes

The mean stayed stable across all methods as expected, ranging from 19.67 to 19.79 degrees Celsius. The minimum converged toward the mean as N increased, most noticeably for EMA N=500 which had the highest minimum at 19.430 degrees Celsius vs 19.278 for raw.

The max of 21.622 was identical across every method, which points to a spike early in the run before any filter buffer had enough samples to suppress it.

Standard deviation didn't decrease with filtering as expected. The moving median at N=50 was the only method to marginally beat raw (0.2593 vs 0.2594). EMA at N=500 was actually the

worst at 0.3249, likely because the slow-moving large window introduces drift that shows up as variance rather than reducing it.

Deliverable 15: Advantages and Disadvantages of Each Method

Moving Average: Simple to implement and effective at reducing Gaussian noise. The downside is lag: at $N=500$ the output trails real changes by up to 500 samples. Higher N means lower noise but more lag and more memory for the buffer.

Moving Median: Better at handling spikes and outliers than the moving average since one bad ADC reading can't skew the result the way it would an average. The tradeoff is computation cost. Sorting 500 values every sample is noticeably slower and you can see the loop time increase at $N=500$. Good choice when occasional glitches are expected.

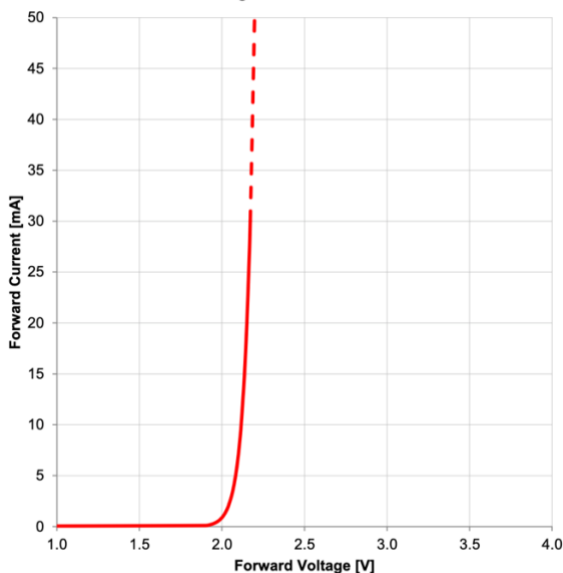
Exponential Moving Average: Only needs two floats of state so memory use is minimal. Responds faster to real changes than a moving average of equivalent N because recent samples are weighted more. The downside is old data is never fully forgotten. In theory higher N should give smoother output, but in practice the $N=500$ EMA had higher standard deviation than $N=50$ (0.3249 vs 0.2728), suggesting the slow-moving large window can introduce drift that shows up as additional variance.

A higher N is worth choosing when the signal changes slowly and you want a cleaner reading. The downsides are more lag, more memory for the moving average and median, and the risk of missing fast real changes in the signal.

Section 7: Basic Actuators

Deliverable 16: LED Forward Voltage

Forward Current vs. Forward Voltage:



From the datasheet curve the forward voltage at around 10-20mA is approximately 2.0-2.1V. What stands out on the curve is how steeply the current rises for small increases in voltage. This is the diode's exponential I-V characteristic, which is why a current-limiting resistor is needed to prevent the LED from drawing too much current.

Deliverable 17: ESP32 GPIO Current Limits

Table 15 – cont'd from previous page

Symbol	Parameter		Min	Typ	Max	Unit
I_{OH}	High-level source current ($V_{DD}^1 = 3.3\text{ V}$, $V_{OH} \geq 2.64\text{ V}$, output drive strength set to the maximum)	VDD3P3_CPU power domain 1, 2	—	40	—	mA
		VDD3P3_RTC power domain 1, 2	—	40	—	mA
		VDD_SDIO power domain 1, 3	—	20	—	mA
I_{OL}	Low-level sink current ($V_{DD}^1 = 3.3\text{ V}$, $V_{OL} = 0.495\text{ V}$, output drive strength set to the maximum)		—	28	—	mA

The GPIO pins can source and sink up to 40mA. With a 330 ohm resistor the LED draws about 3.6mA (1.2V drop across the resistor at 330 ohms), well within the limit. Without the resistor the GPIO pin would try to supply far more than 40mA and would likely be damaged or destroyed.

Deliverable 18: LEDs 1 and 3 On, LED 2 Off

Screenshot of serial monitor with input 101

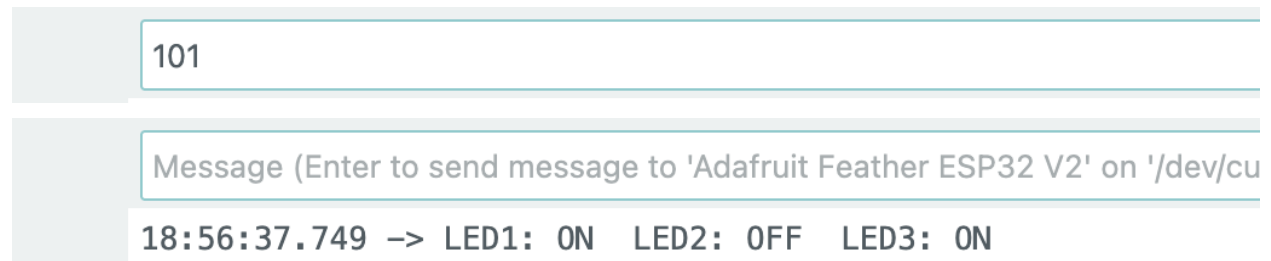
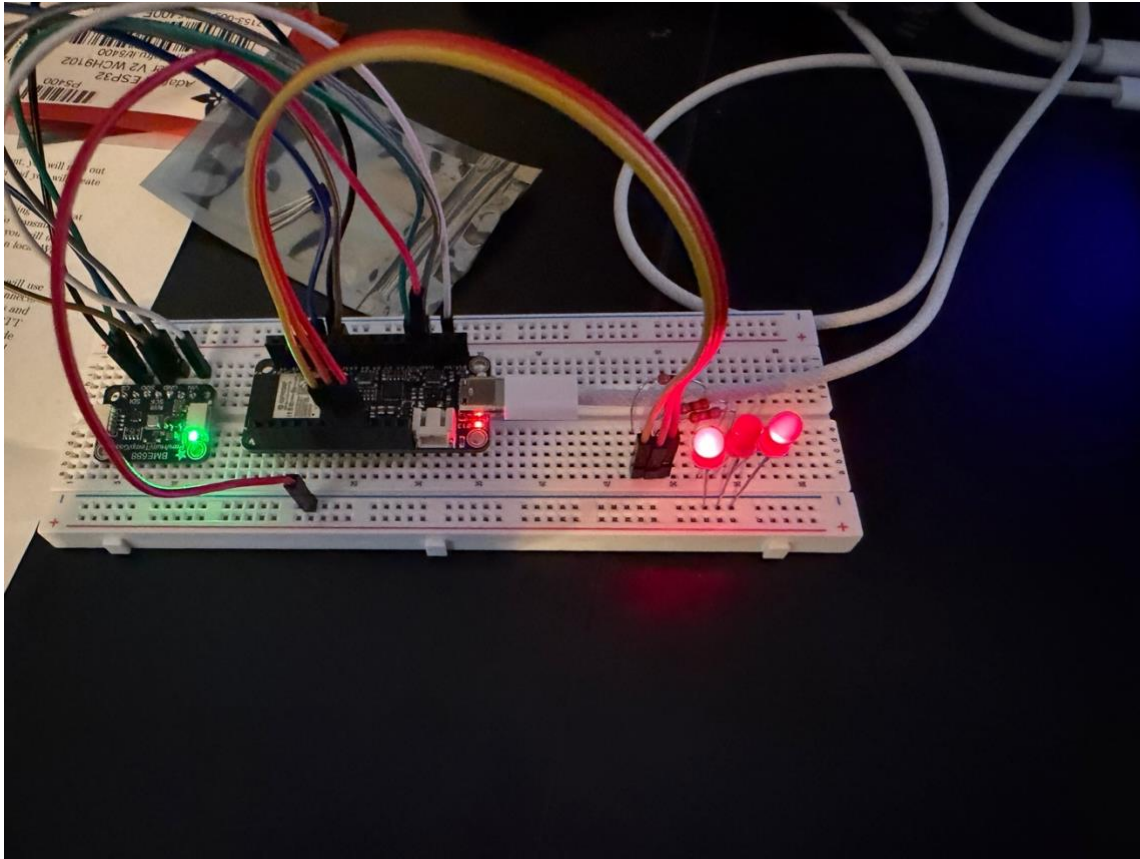


Photo of breadboard showing LED 1 and LED 3 on, LED 2 off



Section 9: Making Decisions and Taking Action

Deliverable 19: Complete System Code

```
1  #include <Wire.h>
2  #include <SPI.h>
3  #include <Adafruit_Sensor.h>
4  #include "Adafruit_BME680.h"
5
6  // ---- Pins ----
7  #define ADC_PIN    A2
8  #define LED_PIN    13
9  #define BUTTON_PIN 38
10
11 #define BME_SCK 5
12 #define BME_MISO 21
13 #define BME_MOSI 19
14 #define BME_CS 26
15
16 Adafruit_BME680 bme(BME_CS, BME_MOSI, BME_MISO, BME_SCK)
17
18 // ---- EMA config ----
19 const float ALPHA = 2.0 / (50 + 1);
20 float emaTMP36 = 0;
21 float emaBME   = 0;
22
23 // ---- Threshold ----
24 const float DELTA_THRESHOLD = 1.5;
25
26 // ---- Timing ----
27 unsigned long lastSampleTime = 0;
28 const unsigned long SAMPLE_MS = 200;
29
30 // ---- State ----
31 bool started    = false;
32 bool lastButton = HIGH;
33
34 void setup() {
35   Serial.begin(115200);
36   analogReadResolution(12);
```

```

34 void setup() {
35     Serial.begin(115200);
36     analogReadResolution(12);
37
38     // Set attenuation once at startup and leave it
39     analogSetPinAttenuation(ADC_PIN, ADC_6db);
40     delay(500);
41
42     pinMode(LED_PIN, OUTPUT);
43     pinMode(BUTTON_PIN, INPUT_PULLUP);
44     digitalWrite(LED_PIN, LOW);
45
46     if (!bme.begin()) {
47         Serial.println("BME688 not found - check SPI wiring!");
48         while (1);
49     }
50
51     bme.setTemperatureOversampling(BME680_OS_1X);
52     bme.setIIRFilterSize(BME680_FILTER_SIZE_0);
53     bme.setHumidityOversampling(BME680_OS_NONE);
54     bme.setPressureOversampling(BME680_OS_NONE);
55     bme.setGasHeater(0, 0);
56
57     Serial.println("Ready - press button to start.");
58 }
59
60 void loop() {
61     // ---- Button debounce ----
62     bool currentButton = digitalRead(BUTTON_PIN);
63     if (lastButton == HIGH && currentButton == LOW) {
64         delay(50);
65         if (digitalRead(BUTTON_PIN) == LOW) {
66             started = !started;
67             if (started) {
68                 // Flush ADC before seeding EMA
69                 for (int i = 0; i < 5; i++) { analogRead(ADC_PIN); delay(10); }

```

```

68 // Flush ADC before seeding EMA
69 for (int i = 0; i < 5; i++) { analogRead(ADC_PIN); delay(10); }
70
71 // Seed EMA from stable reading
72 int dn = analogRead(ADC_PIN);
73 emaTMP36 = ((dn / 4095.0) * 2.1 - 0.5) / 0.010;
74
75 bme.performReading();
76 emaBME = bme.temperature;
77
78 Serial.printf("Seeded: TMP36=%.2fC BME=%.2fC\n", emaTMP36, emaBME);
79 Serial.println("Started.");
80 } else {
81     digitalWrite(LED_PIN, LOW);
82     Serial.println("Stopped.");
83 }
84 }
85 }
86 lastButton = currentButton;
87
88 if (!started) return;
89
90 unsigned long now = millis();
91 if (now - lastSampleTime < SAMPLE_MS) return;
92 lastSampleTime = now;
93
94 // ---- Read TMP36 ----
95 int dn = analogRead(ADC_PIN);
96 float voltage = (dn / 4095.0) * 2.1;
97 float rawTMP36 = (voltage - 0.5) / 0.010;
98
99 // ---- Read BME688 ----
100 if (!bme.performReading()) {
101     Serial.println("BME688 read failed");
102     return;
103 }
104 float rawBME = bme.temperature;
105

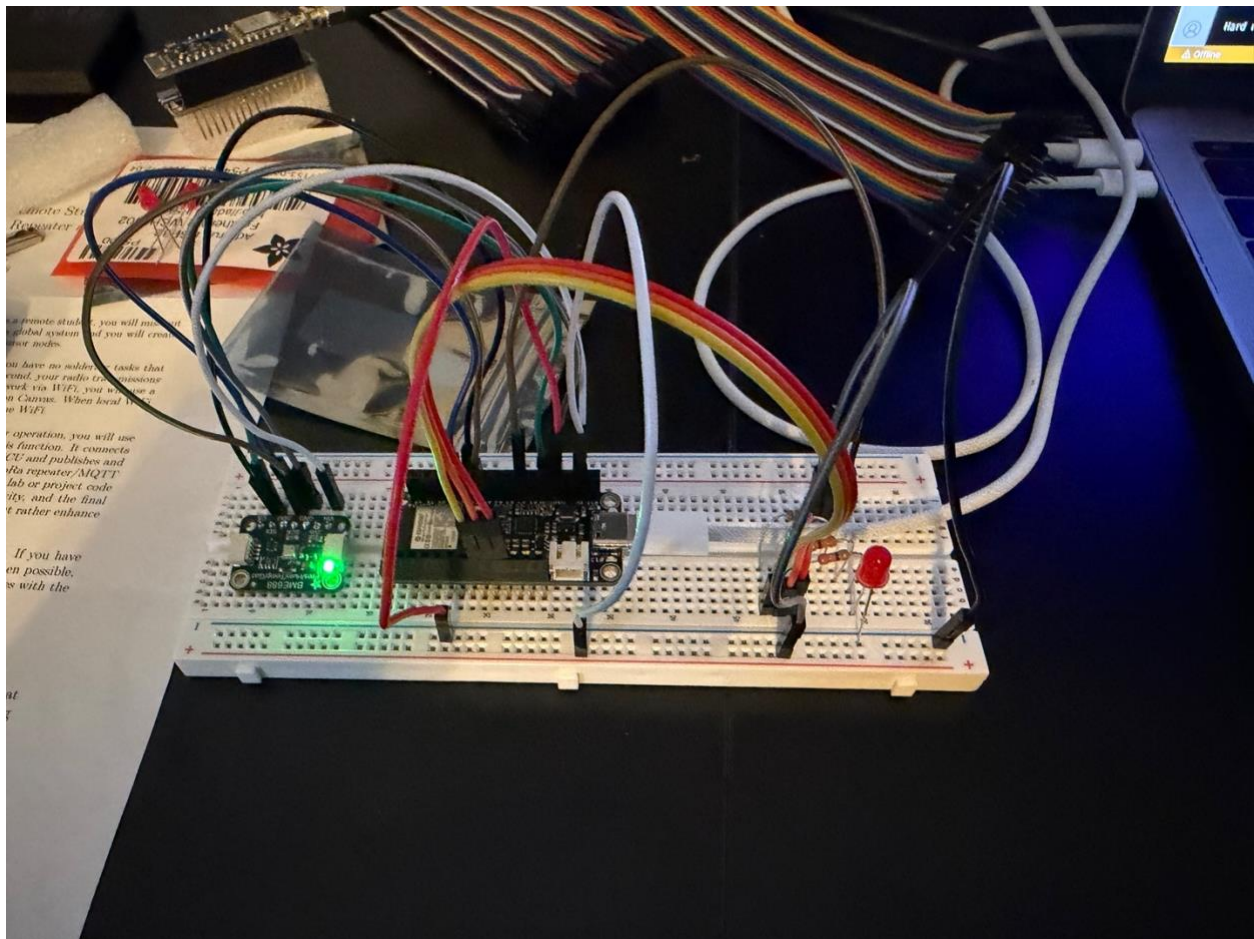
```

```

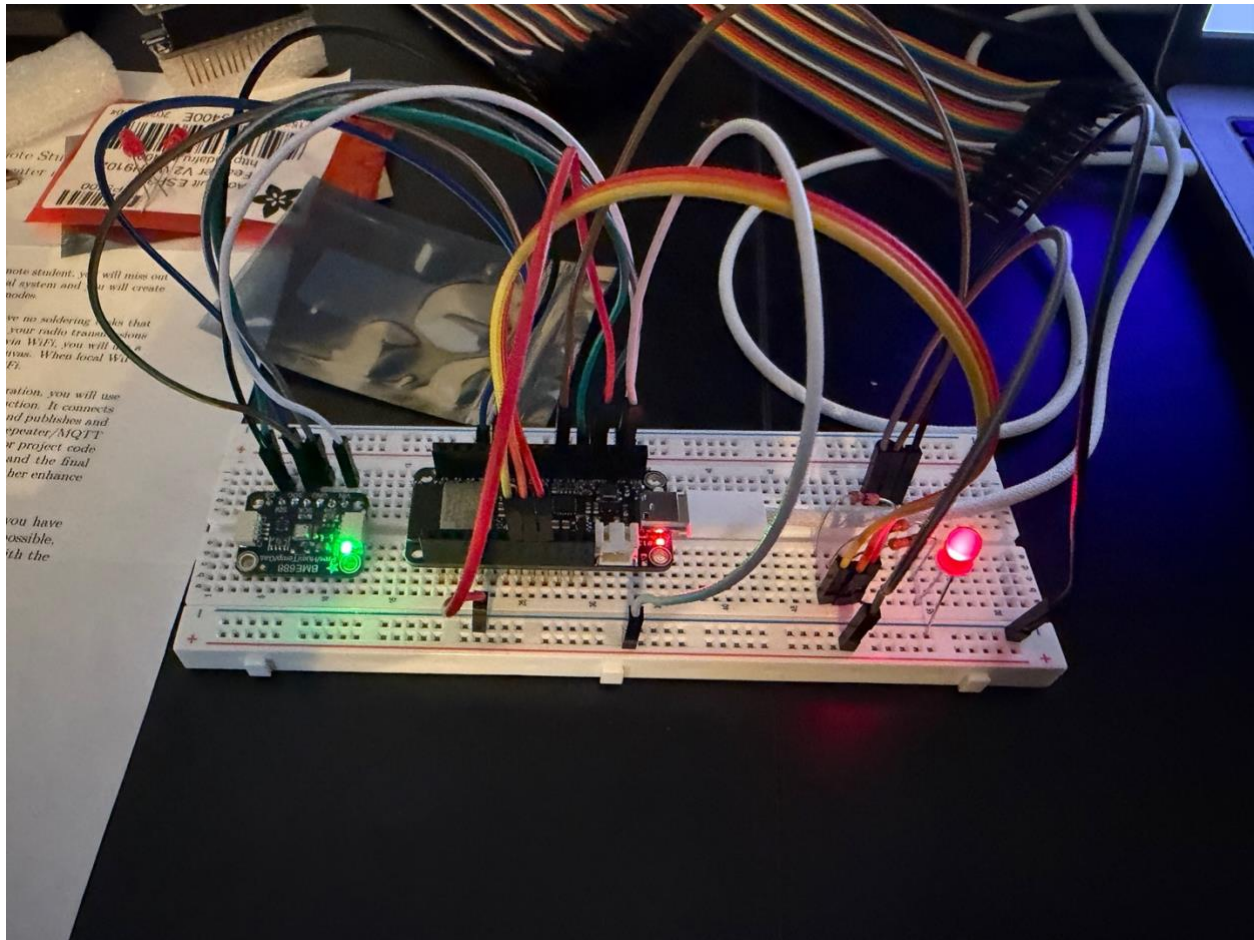
104     float rawBME = bme.temperature;
105
106     // ---- EMA filter ----
107     emaTMP36 = ALPHA * rawTMP36 + (1.0 - ALPHA) * emaTMP36;
108     emaBME    = ALPHA * rawBME    + (1.0 - ALPHA) * emaBME;
109
110     // ---- Compare and actuate ----
111     float delta = emaTMP36 - emaBME;
112     if (delta < 0) delta = -delta;
113
114     bool ledOn = delta > DELTA_THRESHOLD;
115     digitalWrite(LED_PIN, ledOn ? HIGH : LOW);
116
117     Serial.printf("TMP36: %.2fC BME688: %.2fC Delta: %.2fC LED: %s\n",
118                 emaTMP36, emaBME, delta, ledOn ? "ON" : "OFF");
119 }

```

Deliverable 20: System with LED Off



Deliverable 21: System with LED On



To trigger the LED I held a finger on the TMP36 to warm it up. Once the filtered temperature delta between the TMP36 and BME688 exceeded the 1.5 degree threshold the LED turned on.

Deliverable 22: Temperature Delta Between Sensors at Room Temperature

Yes, there was a small offset between the two sensors at room temperature. This is most likely because the BME688 sits close to the ESP32 chip which generates heat during operation, causing it to read slightly warmer than the TMP36. The two sensors also have different calibration tolerances (TMP36 +/- 2 degrees, BME688 +/- 1 degree) so some systematic offset between them is expected even in the same environment.

Deliverable 23: Design Decisions

I used SPI for the BME688 since it was already wired that way from Section 5.

For signal processing I used an EMA with $\alpha = 2/(50+1)$. It only needs two float variables, responds faster than an equivalent moving average, and provides enough smoothing to prevent ADC noise from triggering false LED events.

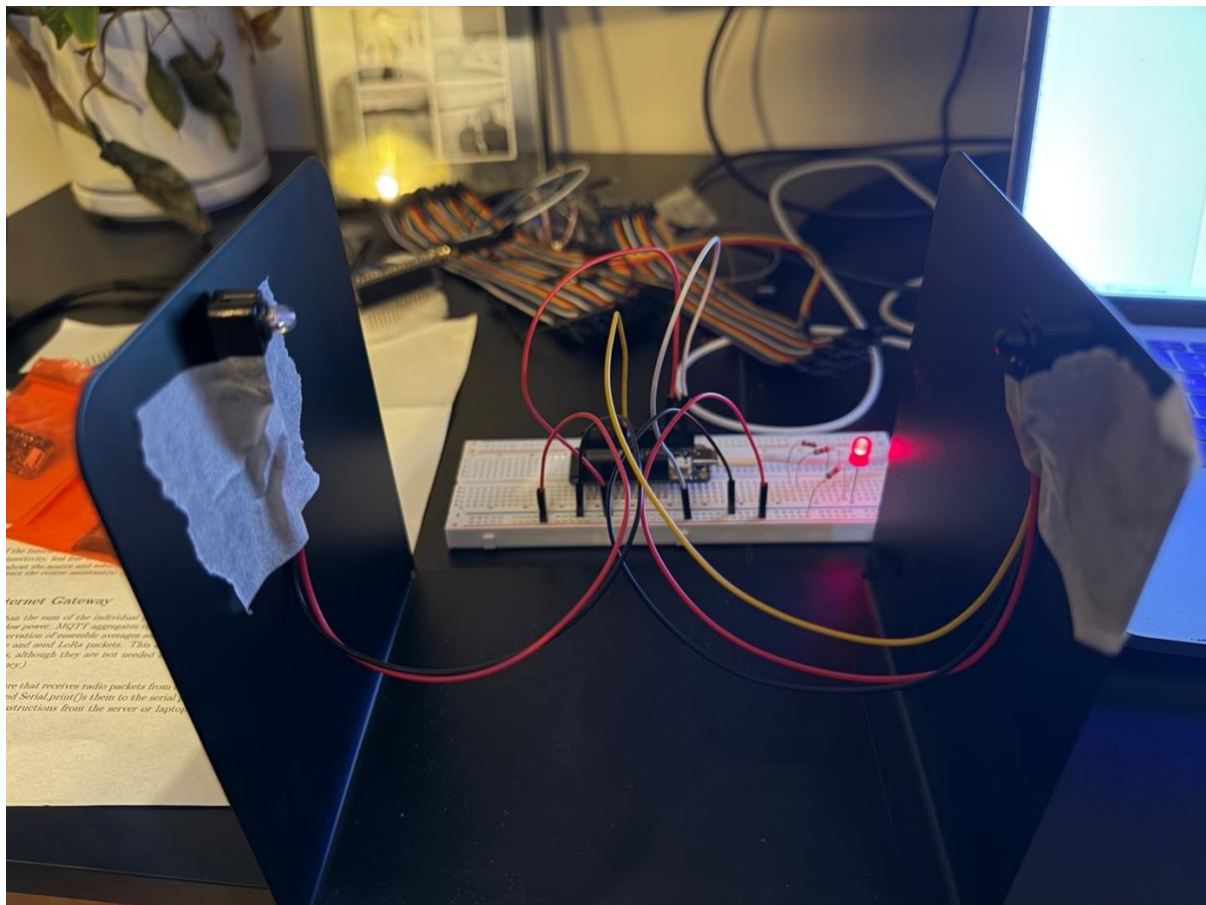
Sensors were sampled alternately each loop iteration. The BME688 takes around 100ms per reading so there's no benefit to batching, and alternating keeps the two timestamps close together.

`analogSetPinAttenuation` was called once in `setup()` with a 500ms settle delay rather than each loop iteration. The EMA was then seeded explicitly in the button handler after a short flush sequence, so the filter starts from a stable value rather than a potentially noisy first reading.

The delta threshold was set to 1.5 degrees Celsius which was above the natural offset between the sensors at room temperature but easy to trigger by touching one sensor.

Section 10: Sensor and Actuation in a Closed Loop

Deliverable 24: IR Sensor Setup



```

20:00:06.105 -> 1
20:00:06.137 -> 1
20:00:06.137 -> 0
20:00:06.137 -> 0
20:00:06.170 -> 0
20:00:06.170 -> 0
20:00:06.170 -> 0
20:00:06.201 -> 0
20:00:06.201 -> 0
20:00:06.201 -> 0
20:00:06.201 -> 0
20:00:06.234 -> 0
20:00:06.234 -> 0
20:00:06.234 -> 0
20:00:06.266 -> 0
20:00:06.266 -> 0
20:00:06.266 -> 0
20:00:06.299 -> 0
20:00:06.299 -> 0
20:00:06.299 -> 0
20:00:06.332 -> 0
20:00:06.332 -> 0
20:00:06.332 -> 1
20:00:06.332 -> 1
20:00:06.332 -> 1

```

Deliverable 25: Role of the 470uF Capacitor

The capacitor acts as a local energy buffer on the 5V rail. When the servo starts or changes speed it draws a large current spike that the USB supply can't deliver fast enough on its own. Without the capacitor this causes the 5V rail to droop and can reset or destabilise the ESP32. The capacitor absorbs the spike and keeps the supply stable. Removing it causes erratic ESP32 behaviour or unexpected resets when the motor moves.

Deliverable 26: Estimated RPM for Three Pulse Widths

Pulse Width (us)	Estimated RPM
1550 (min)	19.0
1650 (mid)	50.0
1750 (max)	65.5

RPM was calculated from the time between rising edges on the IR sensor using: $\text{RPM} = 60 \times 10^6 / \text{interval_us}$

Deliverable 27: Closed-Loop Controller Design

I implemented a proportional controller. On each beam interrupt the interval between rising edges is converted to RPM using $RPM = 60 \times 10^6 / \text{interval_us}$. The error is computed as $e = 60 - \text{rpmMeasured}$ and the pulse width is updated by $K_p \times e$ microseconds with $K_p = 2.0$. The pulse is clamped to $[1550, 1750]$ us after each update. $K_p = 2.0$ gave stable convergence without oscillation. Higher gains caused the RPM to overshoot and hunt around the target. Since the controller only updates on beam interrupts, its update rate naturally slows down at low speeds which helps keep it stable.

Deliverable 28: Video

Video: <https://youtu.be/hdTbqYTmjds>

Code: <https://github.com/ENBayliss44/EE284aLab1>